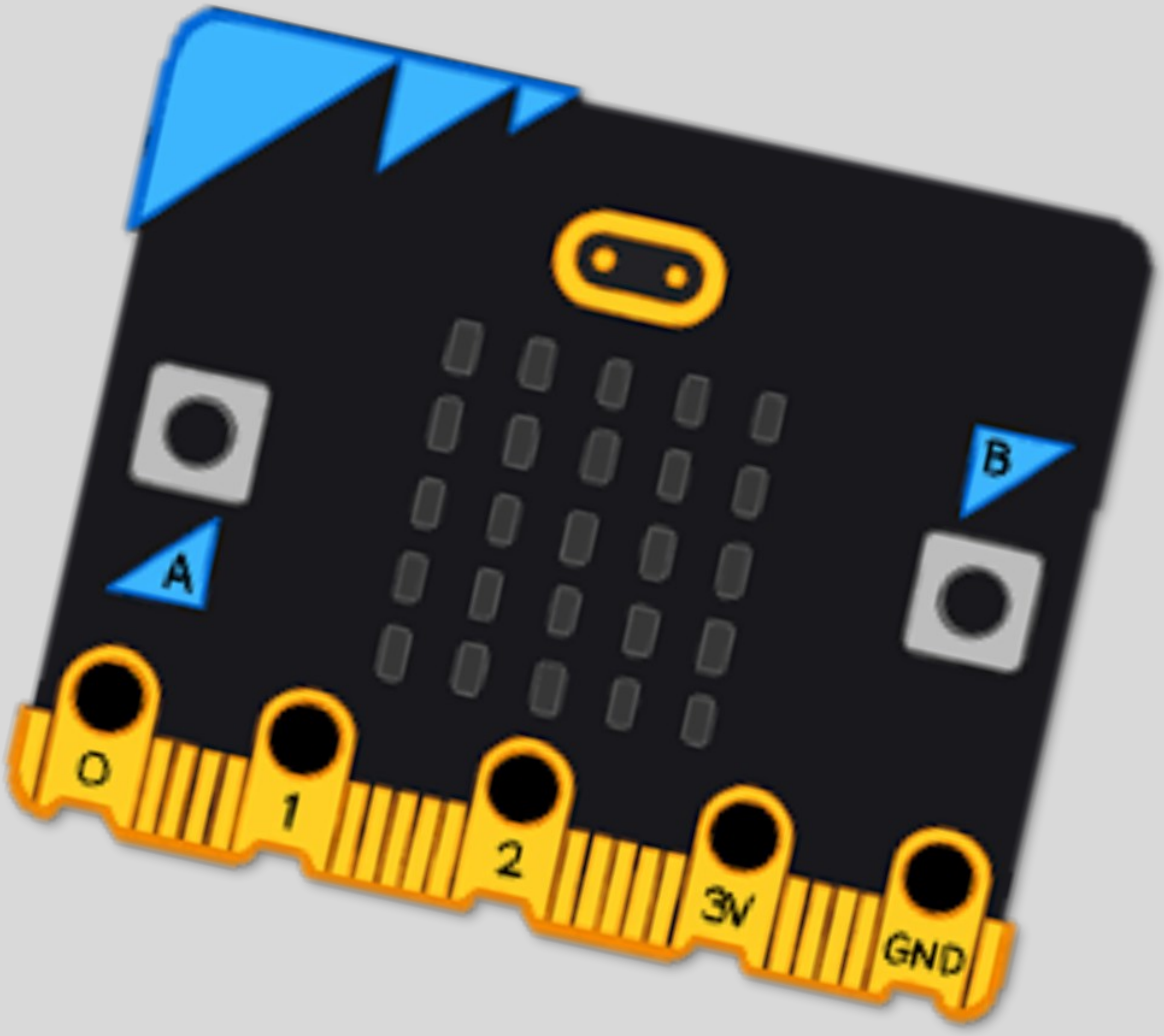
Year 9 ICT
Microbits



Name: _____

Class: _____

What is a BBC Microbit?

The BBC Microbit is a pocket sized computer designed by the BBC. It was designed to encourage students to learn basic coding and programming skills to prepare them for the every growing technology-centred world we live in.

It has multiple sensors, buttons and LEDs to program, allowing for a diverse array of applications – from fitness trackers to games consoles.

A mini-computer board like the BBC Microbit is called a **microcontroller**, basically meaning it is a mini computer with everything on a single board. In order to be considered a microcontroller, it needs to have:

- One or more CPU
- A memory unit (RAM) built in
- Programmable I/O peripherals
- Programmable using a computer language
- All of the above components on a single board

TASK!

Using the internet, write down at least 10 different uses the BBC microbit can be used for:

What does the Microbit have?

The BBC Microbit has several parts, including:

1. 25 programmable LEDs on the front of the board
2. Two programmable buttons (A and B)
3. USB connector
4. Radio & Bluetooth antenna
5. Compass
6. Battery socket
7. Accelerometer
8. Touch sensor
9. 20 pin edge connector to add further devices and sensers, and send commands to other devices

The newer models even have an on-board speaker and microphone!

Activity 1

List Five different uses of the BBC Micro: Bit you have learnt today. You can use the internet if you have forgotten!

1	
2	
3	
4	
5	

Setting up a Microbit

To start using the micro:bit, connect it to a computer via USB.

Visit the micro:bit website (https://makecode.microbit.org) to access the online code editor or download offline development environments.

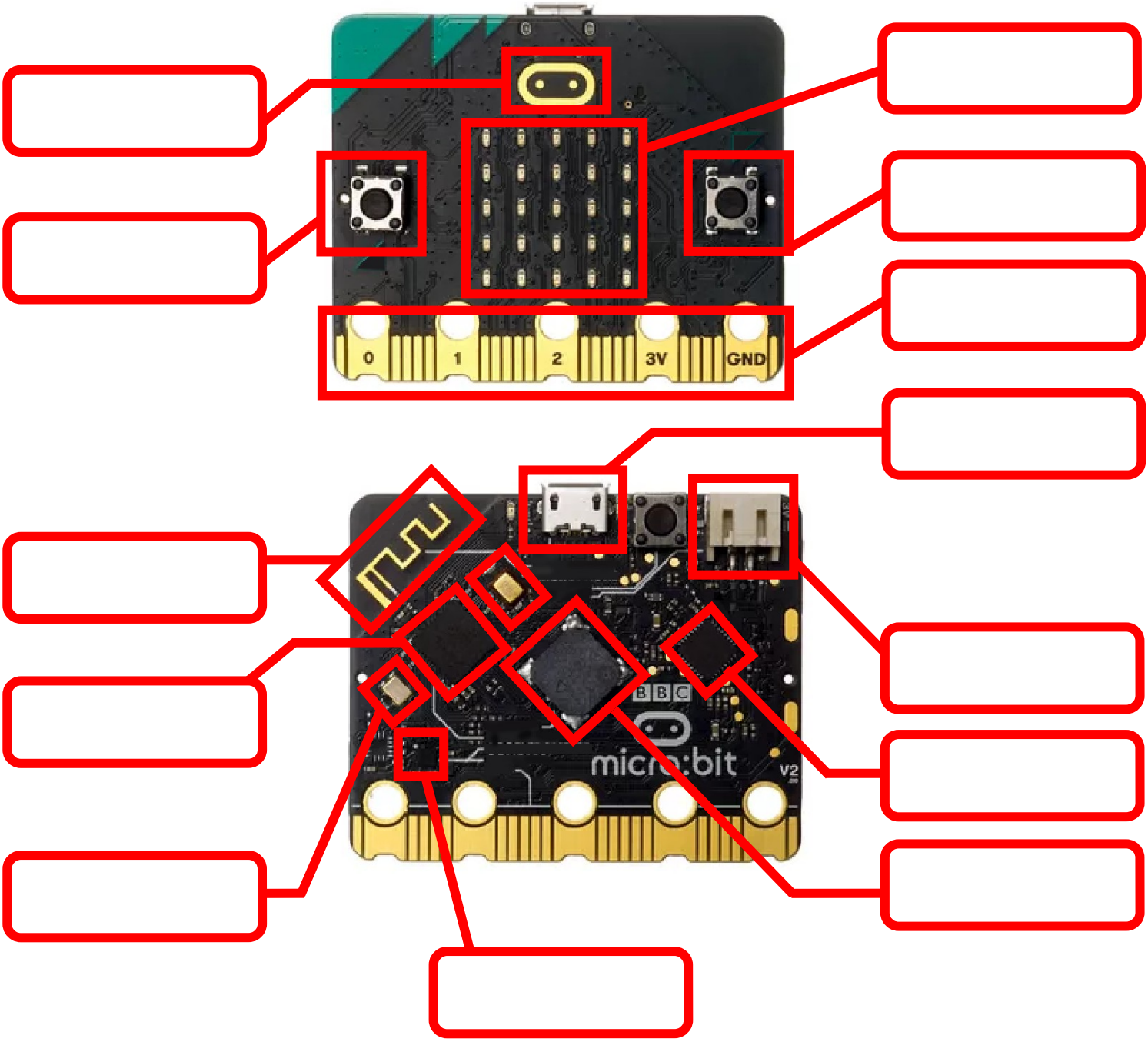
Code for the micro:bit can be written using various programming languages, including MakeCode (block-based) and Python.

Programming concepts such as loops, conditionals, variables, and functions can be applied to create interactive projects.

TASK!

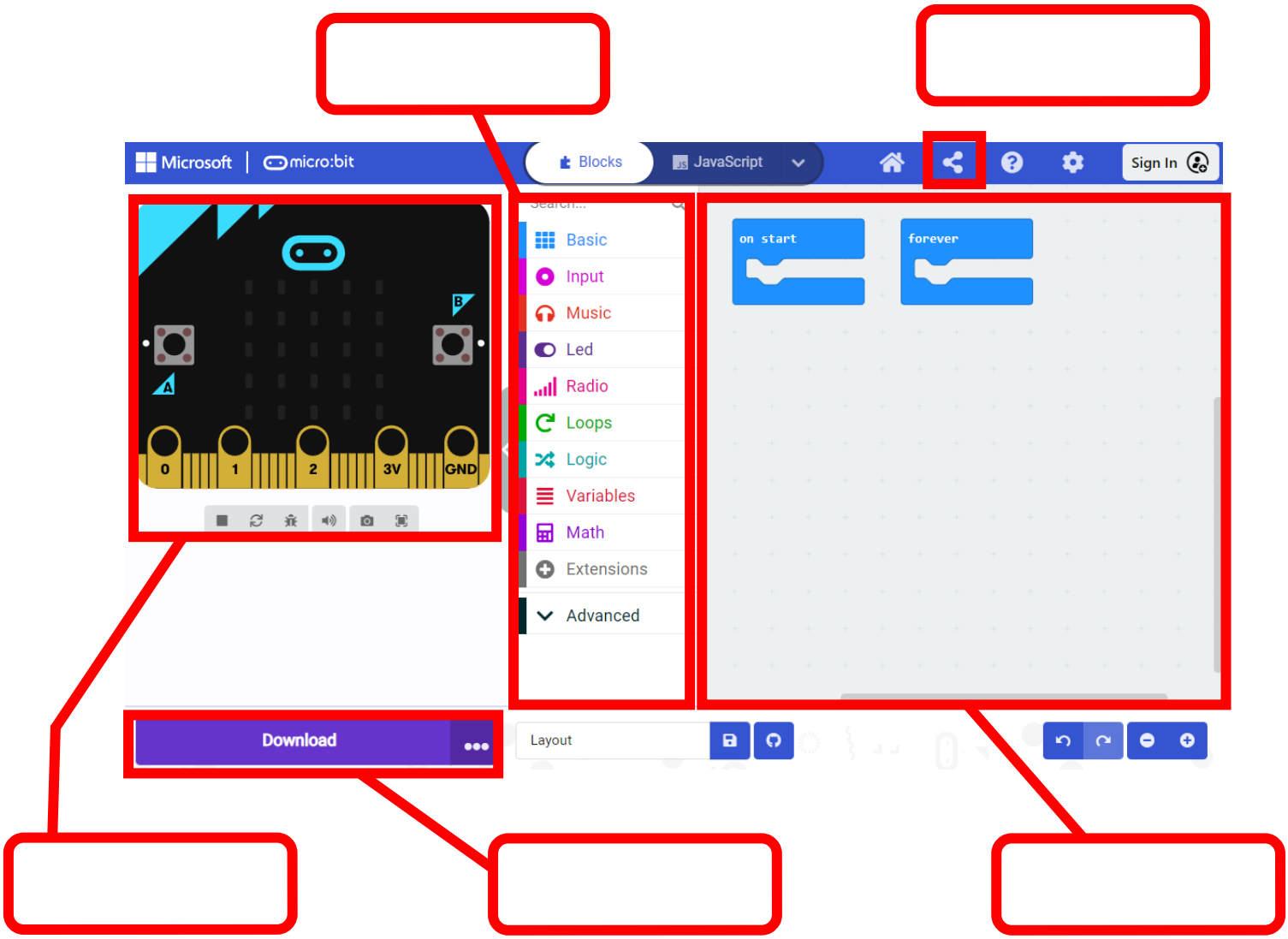
Label the below Microbit Version 2 using the words provided.

25 LEDs Matrix	A Button	B Button	Microphone
20 pin edge connector	Speaker	USB connector	Battery Socket
Compass	Processor	Accelerometer	Touch-sensitive logo
Bluetooth/Radio Antenna			

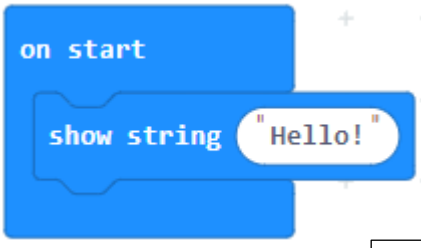


TASK!

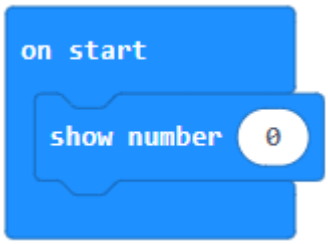
Take the tour on the Microbit website. You can click on the question mark icon at the top right and select tour if you skipped this earlier. Label the



Basic Controls



A string is a sequence of characters, and is used to represent text. So, this 'show string' block is used to tell the Microbit to show a piece of text onto the LEDs of the Microbit.



Another really common data type is numeric data types. These usually fall into two categories:

- Integer
- Double/decimal

An integer is a whole number, and can be positive or negative. A double or decimal number had a decimal point, and can also be positive or negative.

In programming, there are other data types. For example, we use a **char** when representing a single character (letter) or symbol. A **bool** (Boolean) can be used for a true/false or on/off value and is often used to hold a 1 or a 0 value to represent one of these two options.

You will come across other data types across your programming career, but lets stick to the following:

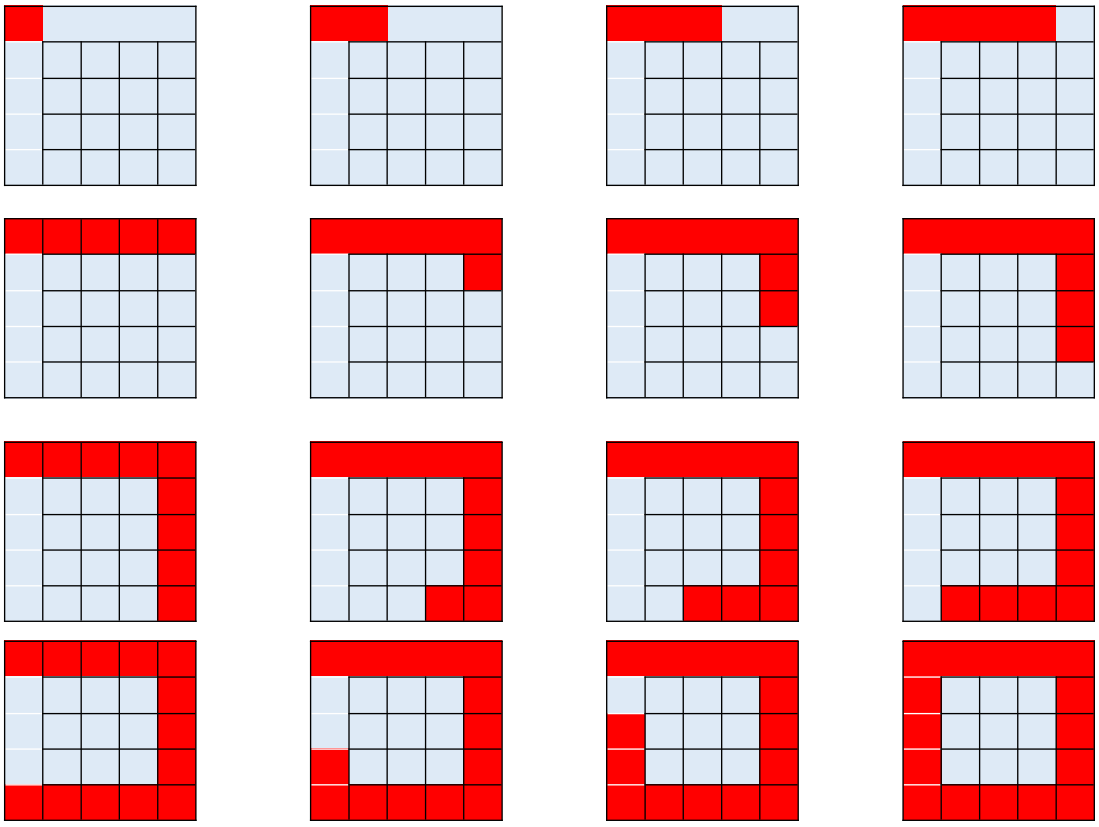
- Integer (int)
- String
- Double
- Character (char)
- Boolean (bool)

Activity 2: A, B, and AB

We are going to create a new program now, so clear the screen of any programming you have completed so far. In this exercise, we are going to create a program that outputs different text when we press different buttons.

The Task

1. When the A button is pressed, the Micro: Bit should say *"You should press button B now"*.
2. When the B button is pressed, the Micro: Bit should then say *"Press both buttons together"*.
3. When both A and B are pressed, the screen should display an animation of:



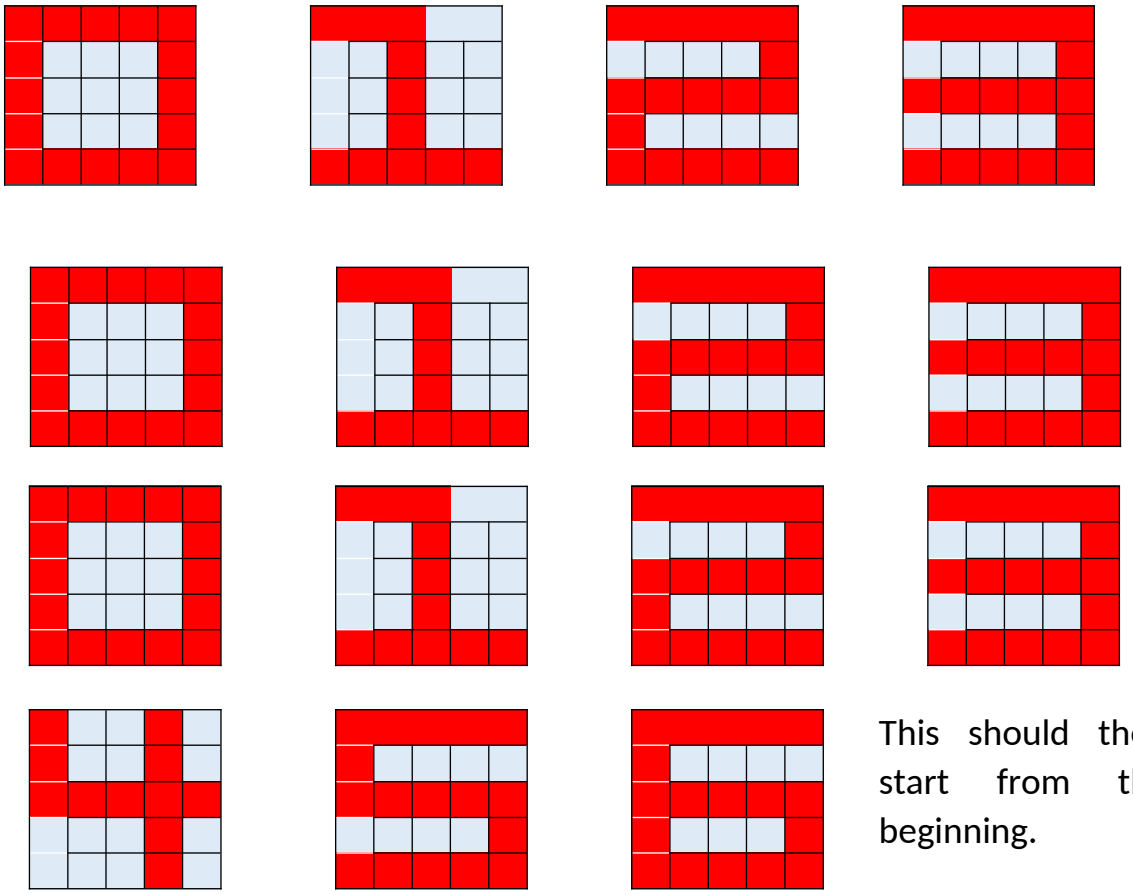
Download this Hex File and call it **buttonBashing**. Don't just save this to the Micro: Bit! Save a copy into My Documents as well to ensure you have this for the future!

Extension Task

Once you have completed Activity 2, you can create your own animation when A and B are pressed. Save this as **buttonBashingExtended** so that you don't overwrite your previous work!

Activity 3: Keeping an Animation Going

Let's create a loop that repeats the numbers '0 1 2 3' **three times** and then says '4 5 6' **once** before going back to repeating the '0 1 2 3' again. This should loop indefinitely.



This should then start from the beginning.

Extension Task: Complete the blanks below:

In My program I have used a _____ loop. This makes the program

_____.

String is the data type we use in Computer Science for variables we are not going to perform any _____ on.

The show string command displays the _____ I have written in the green box on my Micro: bit.

Stuck? Words missing are:

Text Repeat Forever Calculations

Activity 4: Roll the Dice!

Variables

Imagine you have a magic box that can hold different things like toys, snacks, or even secrets. A variable in programming is like that box. It's a special place where we can store information, like numbers or words, and give it a name so we can use it later in our computer program.

Variables are basically these boxes. You give them a name and you can put something in a box (like a piece of text). You can then use the name of the box to reference what is inside of it.

The use of the IF statement is used quite often in programming as it has quite a versatile role. It asks a question – often with a **Boolean** answer.

Task A

Use the internet to find a short definition of the word **Boolean**. Write this in the box below:

Task B

The challenge is to create a program on the Micro: Bit that can be used to replace a dice. In other words, you need to create a dice that 'rolls' a random number between 1 and 6 when you shake the microcontroller.

HINT - You may wish to explore the option in 

the block designer.

Another feature is to look at the variables section of the code. A variable is something that can change or be adapted. So you could have a variable **Age** which may be set to

15. In one year's time this **variable** will change to 16. It's not a set value that cannot change over time. We will go over this further soon.

In your own words, define the term **variable** below:

Save your code as **RollTheDice** in My Documents.

Loops

One of the most common structures we use in programming is loops, or more commonly known as **iteration**s. This allows certain code to be repeated without having to retype it over and over again. There are two types of loops;

- Condition controlled loops
- Count controlled loops

Condition Controlled Loops

Condition-controlled loops, also known as "while loops" or "do-while loops," are structures in programming that repeatedly execute a block of code as long as a specified condition is true.

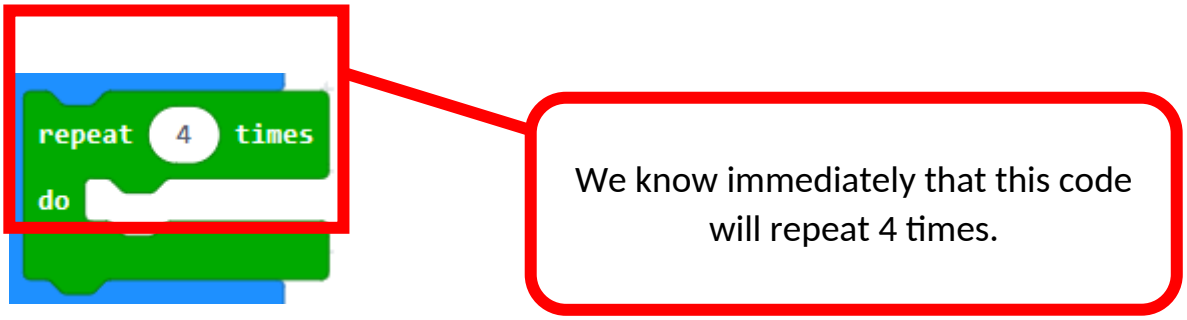
The condition is evaluated before each iteration. If the condition is true, the loop body is executed. After the execution of the loop body, the condition is checked again. If the condition is false, the loop terminates.



Count Controlled Loops

Count-controlled loops, also known as "for loops" or "foreach loops," are structures in programming that iterate a specific number of times.

Count-controlled loops are used when a specific number of iterations is required, or when iterating over elements of a collection or sequence.



Selection and Logic Gates

Logic gates are fundamental building blocks of digital circuits used in computers and electronic devices. They perform basic logical functions based on binary inputs (0 or 1) and produce a binary output.

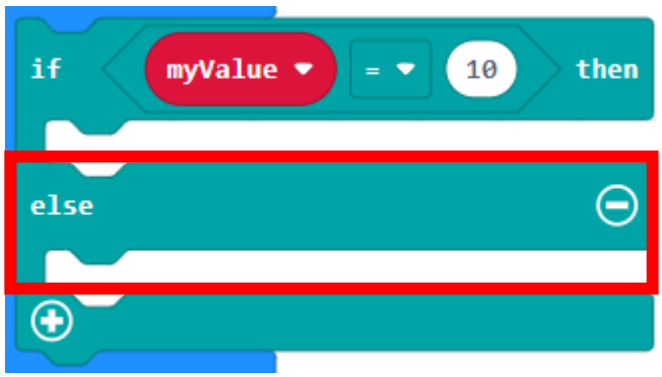
When programming, we use logic gates to decide if a piece of code should run or not. This is a programming concept of **selection**, which uses an **if** statement.



A piece of code will only run **if** a condition is true.

IF statements are usually paired with **else** statements.

This means that if a condition **is not true**, then you can make something else happen.



Logic Gates

Logic Gates are fundamental to use in developing useful programs. These are commonly known as logical operators, and come in the form of AND, OR, NOT, etc.

In electronics, they can perform complex operations in digital circuits such as adders, multiplexers and flip-flops. These are all needed to not only program, but to store, read and calculate on a computer.

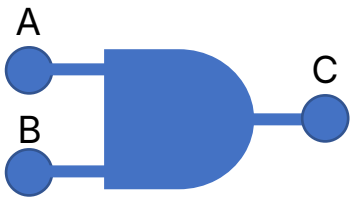
The common logic operators are:

- AND Gates
- OR Gates
- NOT Gates

These usually take in some electrical wires, and these 'gates' determine if the electrical current passes onto the output wire.

The AND Gate

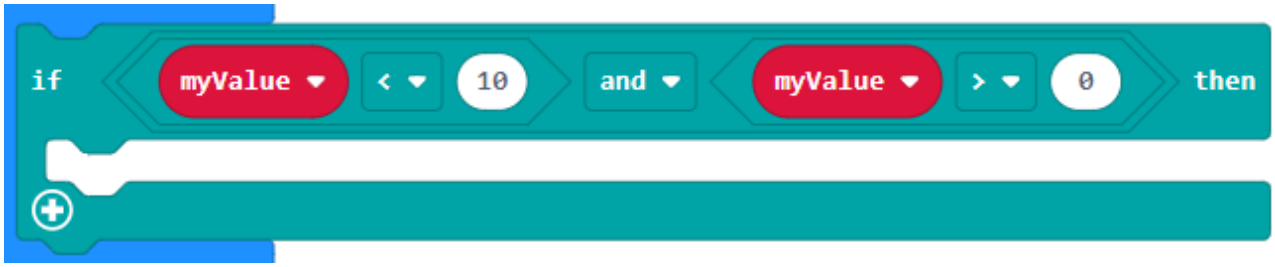
In the AND gate, all inputs must be true in order for it to output as true.



The AND Symbol Notation

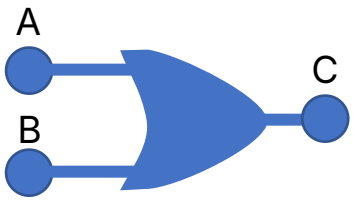
AND Logic Gate		
A	B	C
0	0	
0	1	
1	0	
1	1	

In coding, this would look like this. In the below example, you my want to check that a value is less than 10 but also should be bigger than 0 before you do something.



The OR Gate

In the OR gate, only one input needs to be true in order for it to output as true.

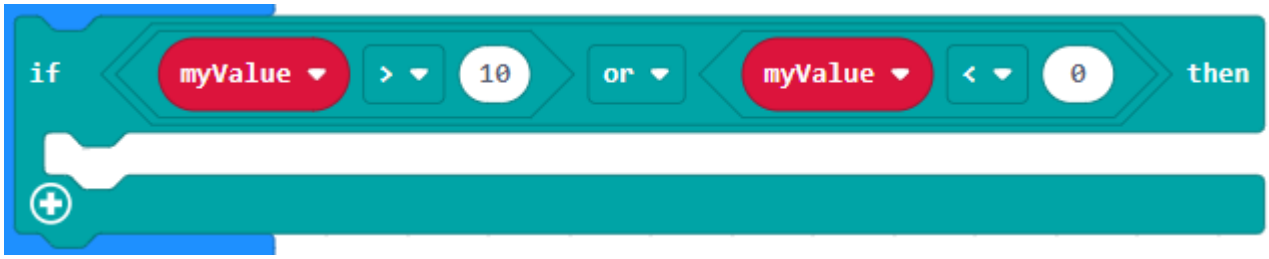


The OR Symbol Notation

OR Logic Gate		
A	B	C
0	0	
0	1	
1	0	
1	1	

In the code below, if myValue is bigger than 10 **OR** myValue is less than 0, you may want to do something.

You may feel this is very similar to what the AND gate code was doing earlier, and you would be right. However, that's only in this example. It could also have been two very unrelated values. But as long as one side of the OR statement is true, then the code within the if statement will run.



Have a look to see if you can find work out the truth table for NAND ('not and'). Create a truth table in the space below.

NAND Gate		
Input A	Input B	Output
True	True	
True	False	
False	True	
False	False	

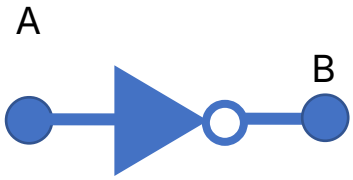
TASK

Write a simple code that demonstrates the OR logic using two buttons (A and B) as inputs.

If either button A or button B is pressed, the micro:bit should display a specific message indicating that the condition is true.

The NOT Gate

The NOT gate is a bit different to other gates, as it only has one input. It inverts its input, so logically it would be like saying “If it WAS true, make the output false”.

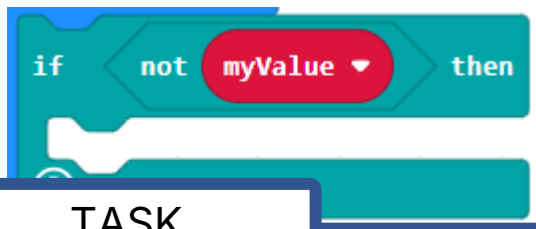


The NOT Symbol Notation

NOT Logic Gate	
A	B
0	
1	

This can be used to test if a condition is NOT true. So, to explain the below:

- The IF statement is used to see if a piece of code should execute or not.
  - It will run if a condition is true.
  - It will not run if the condition is false.
- The below checks if myValue is true or false. It **inverts the variable**, and then tests if the code should do something.



TASK

Write a simple code that demonstrates the NOT logic using a button as input.

If the button is not pressed (input is 0), the micro:bit should display a specific message indicating that the condition is true (output is 1).

Comparison Operators

Logical operators are used constantly in programming, mathematics and science. This is because there are many reasons you need to compare two items. The common operators are:

-	Greater than	>	Returns true if the left operand is greater than the right operand
-	Less than	<	Returns true if the left operand is less than the right operand.
-	Greater than or equal to	>=	Returns true if the left operand is greater than or equal to the right operand.
-	Less than or equal to	<=	Returns true if the left operand is less than or equal to the right operand.
-	Equal to	==	Returns true if the operands are equal.
-	Not equal to	!=	Returns true if the operands are not equal.

Comparison operators are commonly used in conditional statements (if, else, while, for) to make decisions based on the comparison result.

They are also used in loops and sorting algorithms to determine the order of elements. Understanding and mastering comparison operators is fundamental for effective programming and algorithm design.

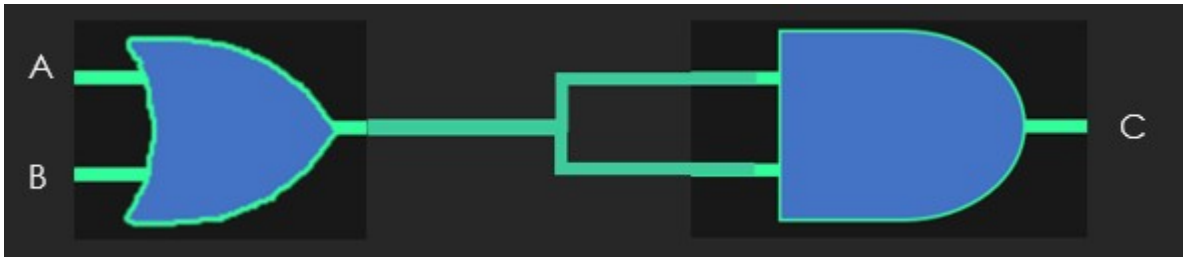
TASK

Write a simple code that demonstrates the use of comparison operators using buttons as inputs and LEDs as outputs.

If button A is pressed and button B is not pressed (&& <), display a smiley face

Extension Task

Look at the logic gates below. What does the A and B need to be set to to make C read as **true**?



A:_____

B:_____

Glossary

TASK

Term	Meaning
Array	
LED	
Logic Gates	
Loops	
Variable	
Accelerometer	

TASK

Using your understanding of logic gates, complete the questions below.

Q1. Complete the following truth table.

P	Q	R = P AND Q	Q OR R
0	0	0	
0	1		
1	0		
1	1	1	

Q. Which of the following truth tables is the correct truth table for NOT(P AND Q)?

A

P	Q	P AND Q	NOT(P AND Q)
0	0	1	1
0	1	1	0
1	0	0	1
1	1	0	0

B

P	Q	P AND Q	NOT(P AND Q)
0	0	0	1
0	1	1	1
1	0	0	0
1	1	1	0

C

P	Q	P AND Q	NOT(P AND Q)
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

D

P	Q	P AND Q	NOT(P AND Q)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Q. Which of the following truth tables is the correct truth table for D=NOT(C AND B)?

A

A	B	C = NOT (B)	D = NOT(C AND B)
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

B

A	B	C = NOT (B)	D = NOT(C AND B)
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	1

C

A	B	C = NOT (B)	D = NOT(C AND B)
0	0	1	1
0	1	0	1
1	0	1	1
1	1	0	1

D

A	B	C = NOT (B)	D = NOT(C AND B)
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	0

Q. Which of the following tables gives the correct output for (P OR Q) AND R?

A

P	Q	P OR Q	R	(P OR Q) AND R
0	0	1	1	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	1

B

P	Q	P OR Q	R	(P OR Q) AND R
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	1	1

C

P	Q	P OR Q	R	(P OR Q) AND R
0	0	0	1	1
0	1	1	1	1
			1	1

D

P	Q	P OR Q	R	(P OR Q) AND R
0	0	0	1	0
0	1	1	1	0
1	0	1	1	0

TASK

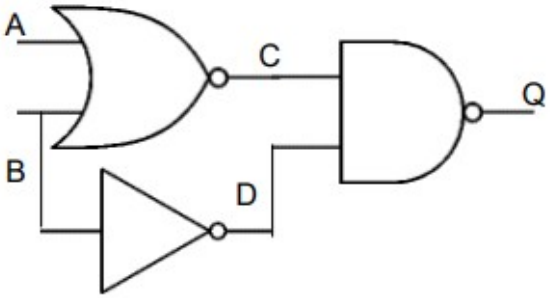
There are other logic gates outside of AND, NOT and OR. Use the internet to research other logic gates including NAND, NOR and XOR.

Use the space provided to create a truth table for each logical operation.

TASK

The questions below show a logic circuit and its incomplete truth table. Complete the below truth tables.

Q1.



A	B	C	D	Q
	0			
	1			
	0			
	1			

Q2.

A	B			D
0	0			
0	1			
1	0			
1	1			

